



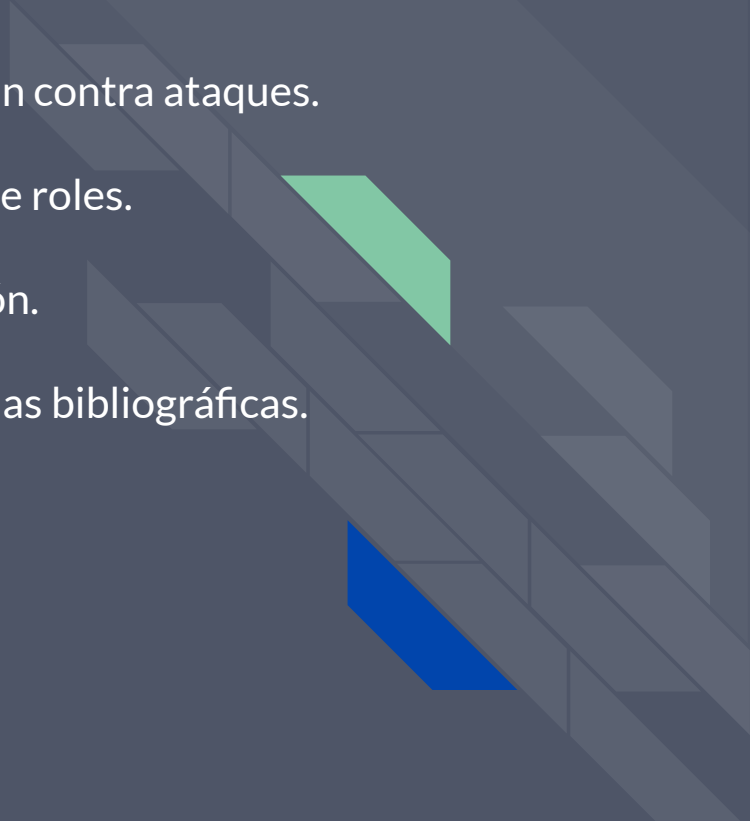
AUTENTICACIÓN DE USUARIOS Y CONTROL DE ACCESO CON SPRING SECURITY

Realizado por:

- Alberto Martínez Lombardo
- José Alejandro Torres Sáez

Curso 2024/2025
Desarrollo de Aplicaciones Web

ÍNDICE

1. Introducción.
 2. Configuración básica.
 3. Autenticación de usuarios.
 4. Control de acceso.
 5. Protección contra ataques.
 6. Gestión de roles.
 7. Conclusión.
 8. Referencias bibliográficas.
- 

1. INTRODUCCIÓN

Problema a tratar

Implementar Spring Security en un proyecto para que la aplicación sea más segura.

Conceptos a aplicar

- Autenticación de usuarios.
- Control de acceso.
- Protección contra ataques.
- Gestión de roles.

1. INTRODUCCIÓN

¿Qué es Spring Security?

Spring Security es un framework de seguridad para aplicaciones web basadas en Java.

¿Para qué sirve Spring Security?

Spring Security se utiliza para aumentar la seguridad de nuestra aplicación web, proporcionándonos **autenticación** (verificar quién eres) y **autorización** (verificar qué puedes hacer) tanto para la **protección de datos** como para el **control de acceso** a nuestra aplicación.

2. CONFIGURACIÓN BÁSICA

01 Inyección de dependencia

02 Clase de configuración

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    UsuarioDAO usuarioDAO;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception { ... }

    @Bean
    public UserDetailsService userDetailsService() { ... }

}
```

3. AUTENTICACIÓN DE USUARIOS

01 loginPage

Página del formulario de login

02 loginProcessingUrl

Envía credenciales para autenticarse

03 failureUrl

Autenticación fallida

04 defaultSuccessUrl

Autenticación exitosa

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .formLogin( FormLoginConfigurer<HttpSecurity> form -> form
            .loginPage("/login.jsp")
            .loginProcessingUrl("/loginUser")
            .failureUrl( authenticationFailureUrl: "/login.jsp?error=true")
            .defaultSuccessUrl( defaultSuccessUrl: "/eventos/listadoEventos", alwaysUse: true)
            .permitAll()
        )
}
```

3. AUTENTICACIÓN DE USUARIOS

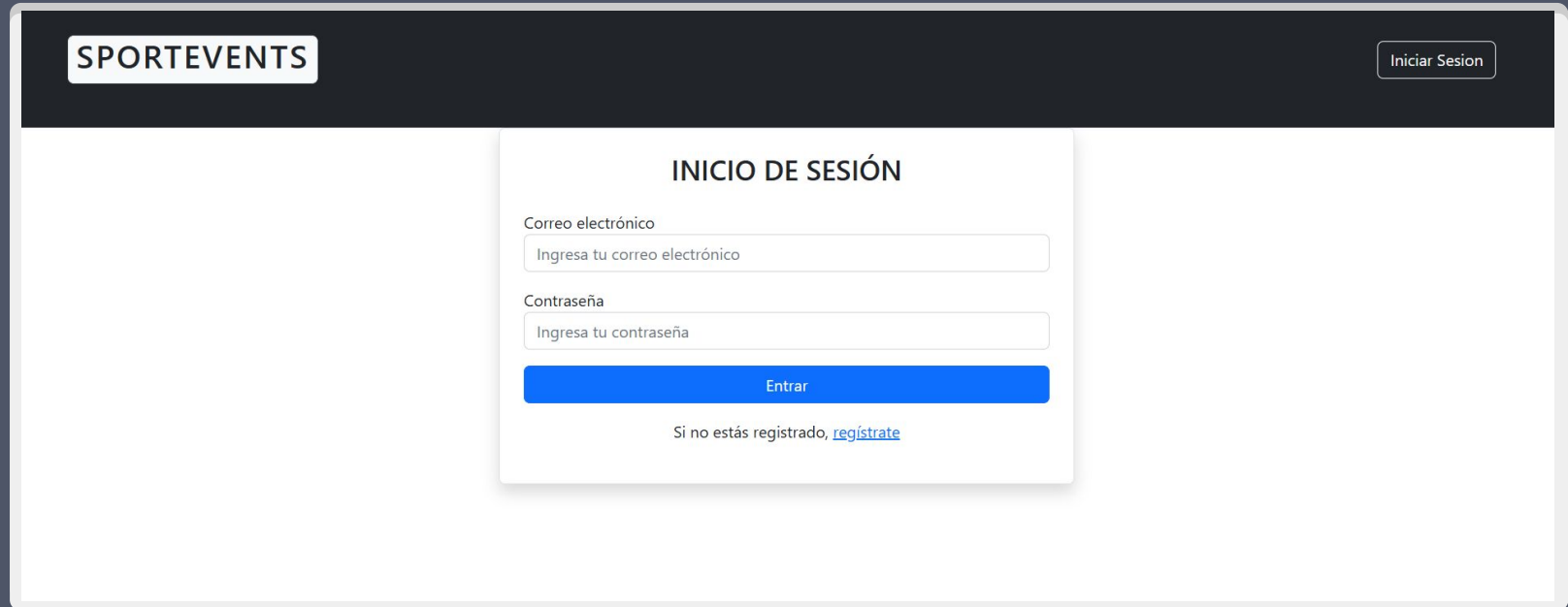
Formulario por defecto

Please sign in

Sign in

3. AUTENTICACIÓN DE USUARIOS

Formulario personalizado



The image shows a computer monitor with a dark grey bezel. On the screen, there is a login interface for 'SPORTEVENTS'. The interface has a dark header bar with the brand name 'SPORTEVENTS' on the left and a 'Iniciar Sesión' button on the right. The main content area is white and features a centered login form titled 'INICIO DE SESIÓN'. The form includes two input fields: 'Correo electrónico' with the placeholder 'Ingresa tu correo electrónico' and 'Contraseña' with the placeholder 'Ingresa tu contraseña'. Below these fields is a prominent blue 'Entrar' button. At the bottom of the form, there is a link that says 'Si no estás registrado, [regístrate](#)'.

SPORTEVENTS

Iniciar Sesión

INICIO DE SESIÓN

Correo electrónico

Ingresa tu correo electrónico

Contraseña

Ingresa tu contraseña

Entrar

Si no estás registrado, [regístrate](#)

3. AUTENTICACIÓN DE USUARIOS

Validación de email

- Busca el email (clave primaria de usuario) introducido en el login.
- Comprueba si está registrado.
- Si no, lanzará una excepción y se mostrará un mensaje de error.

```
@Bean
public UserDetailsService userDetailsService() {
    return String username -> {
        Usuario usuario = usuarioDAO.buscaUsuario(username);
        if (usuario == null) {
            throw new UsernameNotFoundException("Usuario no encontrado");
        }
    }
}
```

3. AUTENTICACIÓN DE USUARIOS

Validación de contraseña

- Utilizamos {noop} para indicar que la contraseña no está encriptada. Entonces, se compara directamente con la introducida en el login.
- Se utilizaría {bcrypt} para contraseñas encriptadas.

```
if (usuario.getEmail().contains("@sportevents.es")) {  
    return new User(  
        usuario.getEmail(),  
        password: "{noop}" + usuario.getContrasena(),  
        Collections.singletonList(new SimpleGrantedAuthority( role: "ROLE_ADMIN"))  
    );  
}
```

4. CONTROL DE ACCESO

Acceso para usuarios no logueados

Todos los usuarios podrán iniciar sesión o registrarse.

```
.authorizeHttpRequests(( AuthorizationManagerRequestMat... auth) -> auth
    .requestMatchers(
        ✓ "/login.jsp",
        ✓ "/usuarios/form",
        ✓ "/WEB-INF/jsp/usuarios/CrearUsuario.jsp"
    ).permitAll()
```

4. CONTROL DE ACCESO

Acceso para usuarios logueados

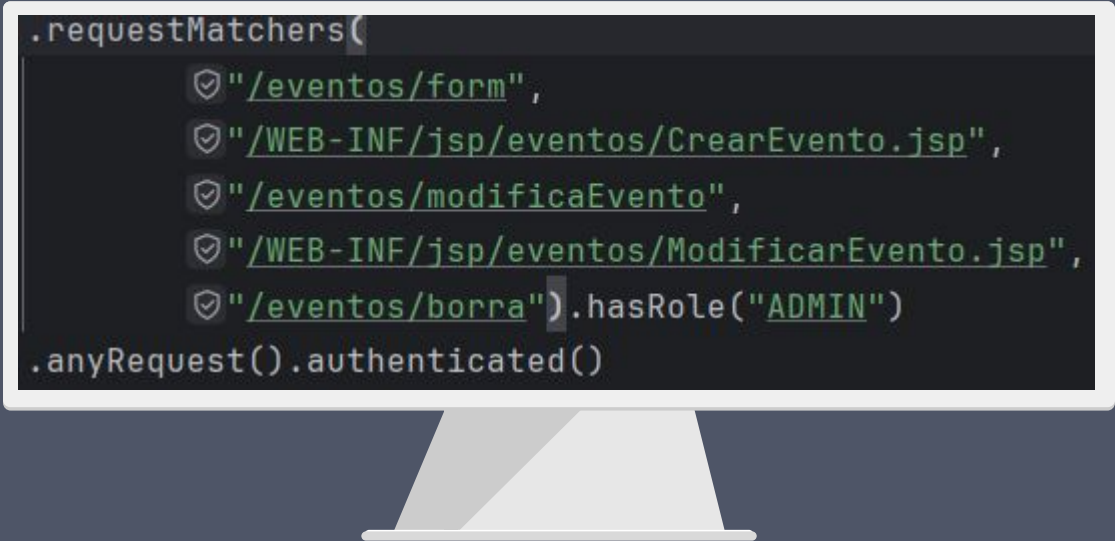
También podrán ver la lista de eventos.

```
.formLogin( FormLoginConfigurer<HttpSecurity> form -> form
    .loginPage("/login.jsp")
    .loginProcessingUrl("/loginUser")
    .failureUrl( authenticationFailureUrl: "/login.jsp?error=true")
    .defaultSuccessUrl( defaultSuccessUrl: "/eventos/listadoEventos", alwaysUse: true)
    .permitAll()
)
```

4. CONTROL DE ACCESO

Acceso para admins

Además podrán crear, modificar y borrar eventos.



```
.requestMatchers(  
    "/eventos/form",  
    "/WEB-INF/jsp/eventos/CrearEvento.jsp",  
    "/eventos/modificaEvento",  
    "/WEB-INF/jsp/eventos/ModificarEvento.jsp",  
    "/eventos/borra").hasRole("ADMIN")  
.anyRequest().authenticated()
```

5. PROTECCIÓN CONTRA ATAQUES

¿Qué es CSRF?

Es un tipo de ataque, denominado falsificación de solicitud entre sitios, donde:

1. El usuario está **logueado** en una aplicación (por ejemplo, su banco).
2. El usuario visita un **sitio web malicioso** sin haber cerrado sesión.
3. El atacante mediante el sitio malicioso **envía una solicitud** al banco para, por ejemplo, transferir dinero, **sin que el usuario lo sepa**.
4. El navegador **envía automáticamente las cookies de sesión** del banco con la solicitud. Entonces, **el servidor no puede distinguir si la solicitud proviene del usuario o de un atacante**.

5. PROTECCIÓN CONTRA ATAQUES

¿Cómo nos protege Spring Security contra CSRF?

1. Spring Security genera un **token CSRF único** para cada formulario, el cual se debe incluir en la solicitud.
2. Este token se verifica en cada envío de formulario para asegurar que la solicitud es legítima.

Campo para el token generado por el servidor:

```
<input type="hidden" name="_csrf" value="ABC123">
```

3. **Cuando se envía un formulario**, Spring verifica que el token en la solicitud coincide con el registrado en el servidor.
4. Si el token no coincide o falta, la solicitud **es rechazada**.

5. PROTECCIÓN CONTRA ATAQUES

CSRF desactivado

```
.logout( LogoutConfigurer<HttpSecurity> logout -> logout
    .logoutUrl("/logout")
    .logoutSuccessUrl("/login.jsp")
    .permitAll()
)
.csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable());
return http.build();
}
```


6. GESTIÓN DE ROLES

Rol de admin

- El email debe contener @sportevents.es.
- Cumplimiento de credenciales.

```
if (usuario.getEmail().contains("@sportevents.es")) {  
    return new User(  
        usuario.getEmail(),  
        password: "{noop}" + usuario.getContrasena(),  
        Collections.singletonList(new SimpleGrantedAuthority(role: "ROLE_ADMIN"))  
    );  
}
```

6. GESTIÓN DE ROLES

Rol de usuario

- Cumplimiento de credenciales.

```
} else {  
    return new User(  
        usuario.getEmail(),  
        password: "{noop}" + usuario.getContrasena(),  
        Collections.singletonList(new SimpleGrantedAuthority(role: "ROLE_USER"))  
    );  
}
```

6. GESTIÓN DE ROLES

Añadimos la librería de etiquetas de Spring Security

```
<%@taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
<sec:authorize access="!isAuthenticated()">
    <a href="<c:url value='/login.jsp'/">" class="btn btn-outline-light">Iniciar Sesion</a>
</sec:authorize>
<sec:authorize access="isAuthenticated()">
    <a href="<c:url value='/logout'/">" class="btn btn-danger">Cerrar Sesion</a>
</sec:authorize>
<sec:authorize access="hasRole('ADMIN')">
    <a class="navbar-brand" href="<c:url value='/eventos/form'/">">Crear evento</a>
</sec:authorize>
```

7. CONCLUSIÓN

Seguridad mediante autenticación 

Seguridad para el acceso mediante urls 

Seguridad ante ataques externos 

Seguridad mediante autorización 

8. REFERENCIAS BIBLIOGRÁFICAS E IAG USADAS

Referencias bibliográficas

<https://www.baeldung.com/spring-security-taglibs>

<https://github.com/persteenolsen/spring-boot-3-mvc-security/blob/main/src/main/java/com/example/accessingdatamysql/config/WebSecurityConfig.java>

<https://www.javaguides.net/2023/04/spring-security-custom-login-page.html>

IAG usada: ChatGPT

El principal uso que se le ha dado ha sido para resolver dudas sobre la configuración de Spring Security en nuestro proyecto y la validación de contraseñas encriptadas y sin encriptar.

8. BIBLIOGRAFÍA PARA PROFUNDIZAR

Página oficial de Spring

<https://docs.spring.io/spring-security/reference/>

Baeldung

<https://www.baeldung.com/security-spring>

Java Guides

<https://www.javaguides.net/p/spring-security-tutorial.html>

¡GRACIAS!

